# CS6650 Midterm Mastery — Part III

*Crashing and Recovering: Resilience Patterns in a Go Product Search API*

Emily Chen | February 27, 2026

## 1. System Overview

This experiment uses the CS6650 Product Search API — a Go HTTP server deployed on AWS ECS Fargate behind an Application Load Balancer (ALB). The server stores 100,000 products in a sync.Map and serves keyword search queries at GET /products/search?q={query}. A simulated AI ranking service (aiRanker) is called after each search to re-rank results before returning them to the client.

| Component | Technology | Configuration |
|---|---|---|
| **Web Server** | **Go net/http** | **Port 8080** |
| **Container** | **Docker (linux/amd64)** | **Alpine-based image** |
| **Orchestration** | **AWS ECS Fargate** | **2 tasks, 256 CPU / 512MB** |
| **Load Balancer** | **AWS ALB** | **Round-robin, 2 targets** |
| **Load Testing** | **Locust (Python)** | **50 users, spawn rate 5** |
| **Monitoring** | **AWS CloudWatch** | **ALB + ECS/ContainerInsights** |

## 2. The Injected Fault

A realistic failure scenario was introduced: the aiRanker dependency occasionally hangs for 30 seconds, simulating an overloaded ML inference endpoint. With a 30% failure probability and no timeout protection in the broken version, goroutines pile up indefinitely waiting for the ranker to respond. Under sustained load from 50 Locust users, this causes cascading failure.

### Broken Version — aiRanker with no protection:

```
func aiRanker(products []Product) []Product {
    if rand.Float32() < 0.30 { // 30% chance of hang
        time.Sleep(30 * time.Second) // blocks goroutine forever
    }
    return products // fast path returns instantly
}

// BUG: no timeout -- goroutines pile up under load
results = aiRanker(results)
```

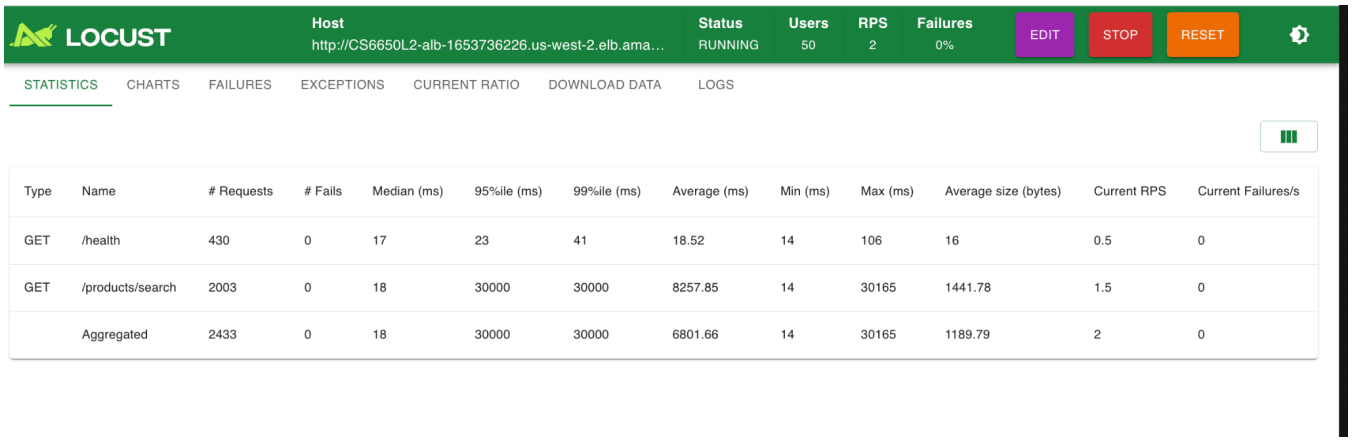
Go's net/http server assigns each incoming request its own goroutine. With 30% of ranker calls hanging for 30 seconds and 50 users sending requests every 0.1–0.5 seconds, dozens of goroutines accumulate holding memory. The ALB's target response time climbs to 7.2 seconds average, throughput collapses to 1.3 RPS, and the P95 latency flatlines at 30,000ms — the exact hang duration.

## 3. Scene 1 — The Broken System

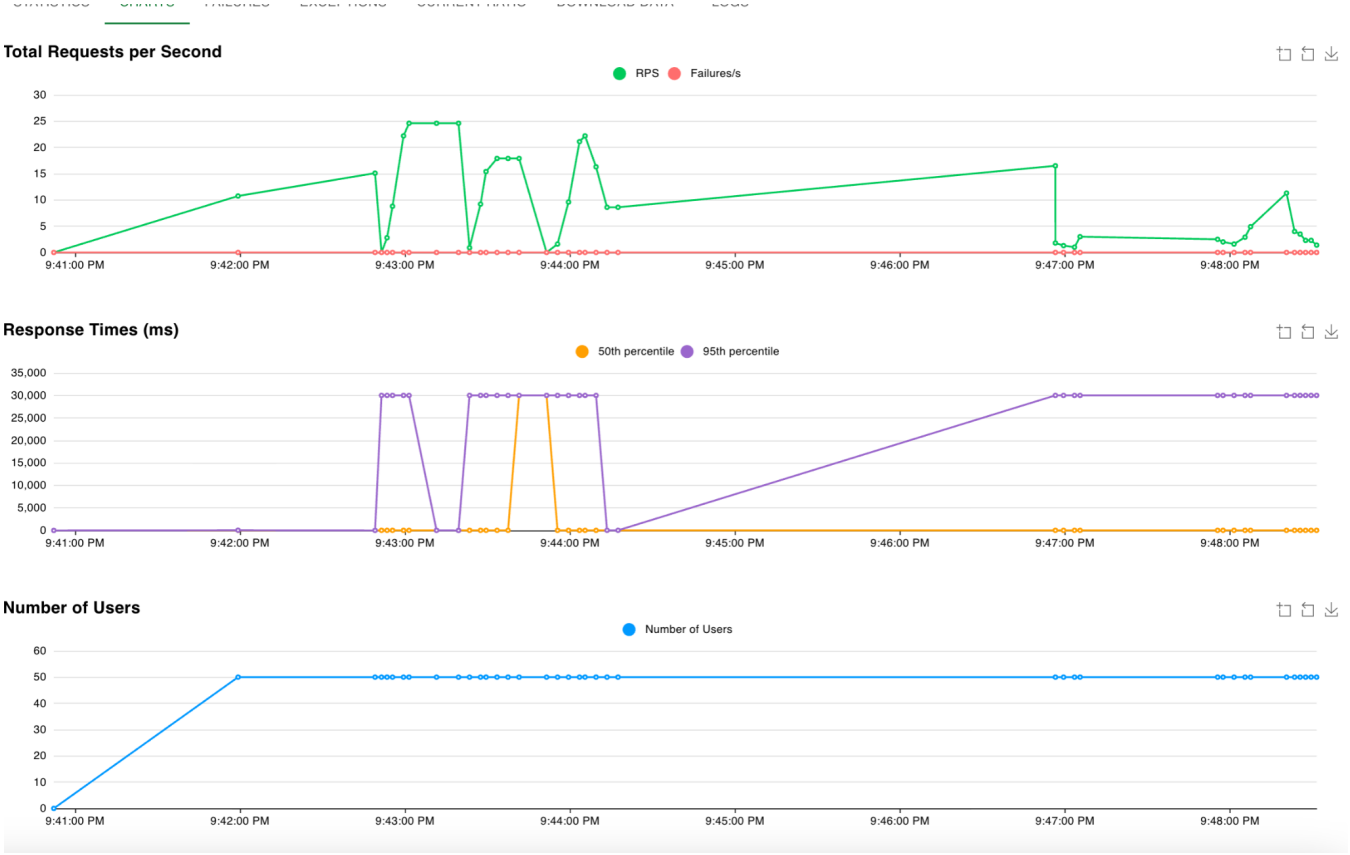
Locust was run against the broken version for 5 minutes with 50 concurrent users. The Locust UI and CloudWatch ALB metrics both captured the degradation clearly.

| Metric | Value | Interpretation |
|---|---|---|
| **Current RPS** | **2** | **Server near-saturated** |
| **Median Latency** | **18 ms** | **Fast 70% path still working** |
| **P95 / P99 Latency** | **30,000 ms** | **Hung goroutines at 30s ceiling** |
| **Average Latency** | **7,982 ms** | **Weighted by 30% hung calls** |
| **ALB TargetResponseTime** | **7.6 seconds avg** | **Measured independently by ALB** |
| **ALB RequestCount / 15min** | **~2,045 requests** | **Severely throttled throughput** |
| **Failure Rate** | **0% HTTP errors** | **Requests eventually return (slowly)** |

***Note: The 0% HTTP failure rate in Locust is deceptive — requests are not being rejected, they are simply taking up to 30 seconds to complete. From a user perspective this is effectively a failure. This illustrates why latency percentiles matter more than raw error counts.***




| Type | Name | # Requests | # Fails | Median (ms) | 95%ile (ms) | 99%ile (ms) | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | Current RPS | Current Failures/s |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| GET | /health | 430 | 0 | 17 | 23 | 41 | 18.52 | 14 | 106 | 16 | 0.5 | 0 |
| GET | /products/search | 2003 | 0 | 18 | 30000 | 30000 | 8257.85 | 14 | 30165 | 1441.78 | 1.5 | 0 |
| | Aggregated | 2433 | 0 | 18 | 30000 | 30000 | 6801.66 | 14 | 30165 | 1189.79 | 2 | 0 |




## 4. The Fix — Three Newman Resilience Patterns

Sam Newman's Building Microservices describes patterns for handling unreliable dependencies. All three were applied in the fixed version, layered in order of increasing scope:

### Pattern 1: Fail Fast (context timeout)

A 500ms context deadline is placed on every aiRanker call. If the ranker does not respond within 500ms, the goroutine abandons the call and returns unranked results immediately. The user still gets a valid response — just without AI ranking. This converts a 30-second hang into a 500ms degraded response.

```
ctx, cancel := context.WithTimeout(context.Background(), 500*time.Millisecond)

defer cancel()

result, err := aiRankerWithTimeout(ctx, products)

if err != nil { return products, "timeout_fallback" } // fail fast
```

### Pattern 2: Circuit Breaker

After 5 consecutive timeouts, the circuit breaker opens and skips the aiRanker entirely for 10 seconds — returning results without even attempting the call. After 10 seconds it goes half-open, sending one probe request. If that succeeds, the breaker closes and normal operation resumes. CloudWatch logs showed the breaker cycling between OPEN and CLOSED states in real time as the 30% fault rate triggered and cleared repeatedly.

### Pattern 3: Bulkhead

A buffered channel semaphore limits concurrent aiRanker calls to 10 at a time. If 10 goroutines are already waiting, new requests are rejected immediately rather than queuing. This prevents a slow dependency from exhausting the entire goroutine pool. The health endpoint reported 433 bulkhead rejections during the test — 433 requests that would have blocked goroutines but were instead served instantly with unranked results.

### Composition — all three patterns applied in sequence:

```
func callRankerSafely(products []Product) ([]Product, string) {

    // Layer 3: Bulkhead — reject if too many concurrent calls

    if !bulkhead.Acquire() { return products, "bulkhead_rejected" }

    defer bulkhead.Release()


    // Layer 2: Circuit Breaker — skip if known-bad

    if !breaker.Allow() { return products, "circuit_open" }


    // Layer 1: Fail Fast — 500ms hard timeout

    ctx, cancel := context.WithTimeout(ctx, 500*time.Millisecond)

    result, err := aiRankerWithTimeout(ctx, products)

    if err != nil { breaker.RecordFailure(); return result, "timeout_fallback" }

    breaker.RecordSuccess()

    return result, "ranked"

}
```

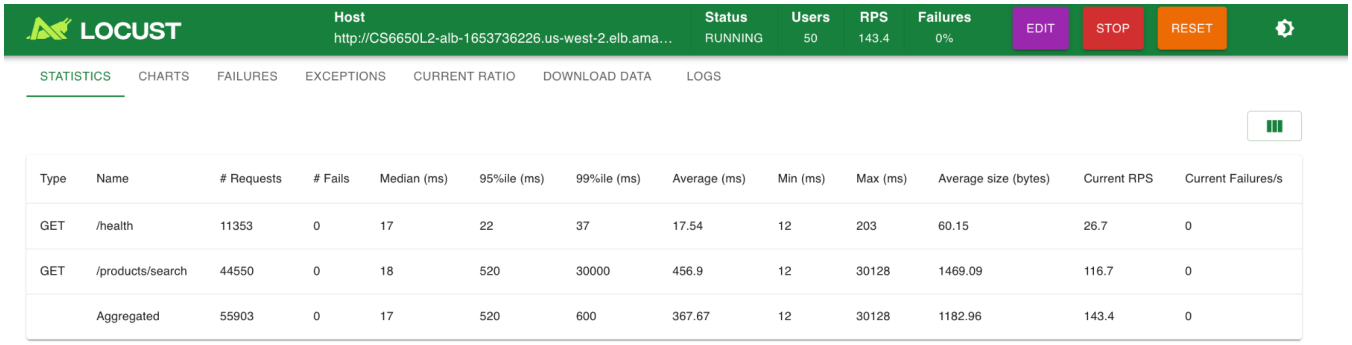
## 5. Scene 3 — The Recovered System

The fixed version was deployed via Terraform while Locust continued running against the same ALB. The transition was captured live on both the Locust Charts tab and CloudWatch metrics.

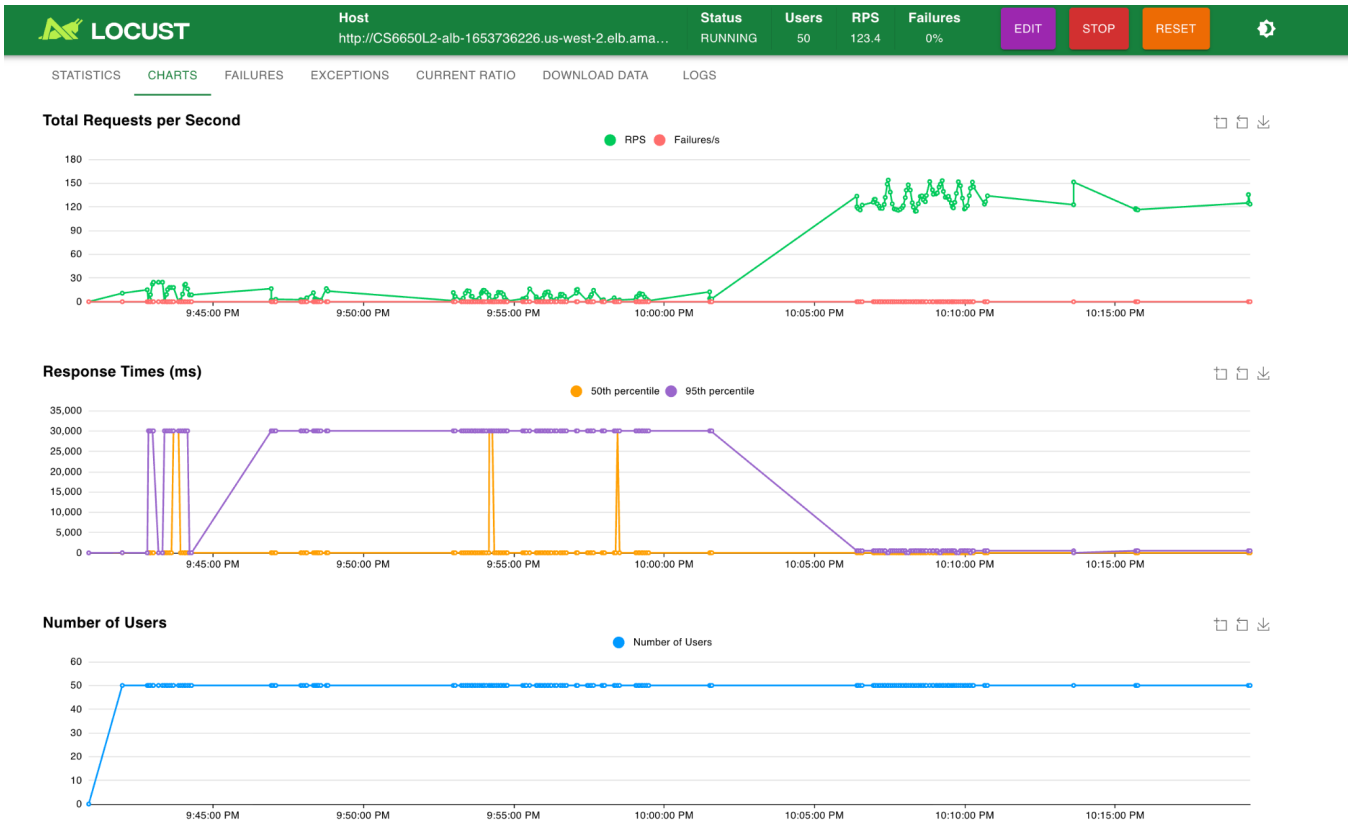
| Metric | Broken Version | Fixed Version |
|---|---|---|
| **Throughput (RPS)** | 1.3 | 143.4 |
| **Median Latency** | 18 ms | 18 ms |
| **P95 Latency** | 30,000 ms | 520 ms |
| **P99 Latency** | 30,000 ms | 600 ms |
| **Average Latency** | 7,982 ms | 457 ms |
| **ALB Req / 15min** | ~2,045 | ~39,320 |
| **ALB Avg Response** | 7.2 seconds | ~0.1 seconds |

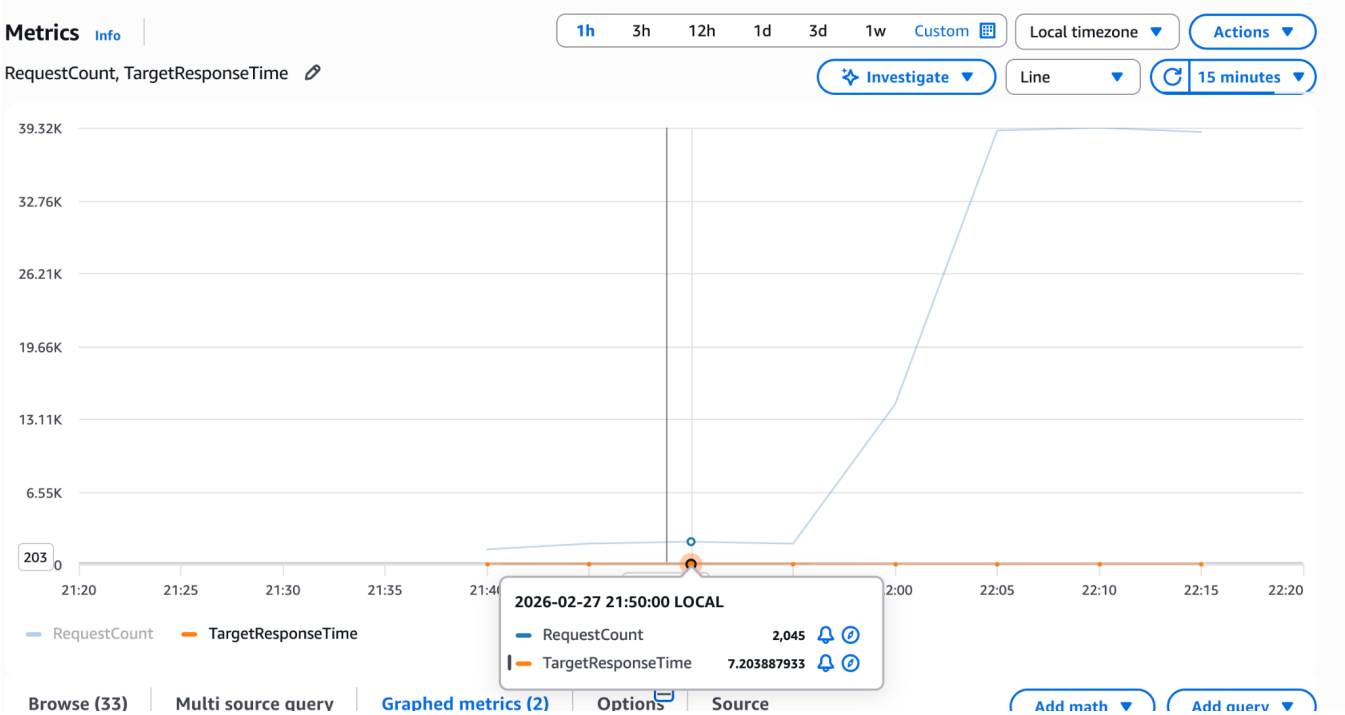
| | | |
|---|---|---|
| **Bulkhead Rejected** | N/A | 433+ requests |
| **Failure Rate** | 0% (timeouts) | 0% |

The most striking result is the 110x improvement in throughput (1.3 → 143.4 RPS) with zero failures. The P95 latency improvement from 30,000ms to 520ms reflects the fail fast pattern: the 30% of calls that previously hung for 30 seconds now time out in exactly 500ms. The CloudWatch ALB chart confirmed this independently — RequestCount per 15-minute window jumped from ~2,045 to ~39,320 at the exact moment the fixed deployment stabilized.




| Type | Name | # Requests | # Fails | Median (ms) | 95%ile (ms) | 99%ile (ms) | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | Current RPS | Current Failures/s |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| GET | /health | 11353 | 0 | 17 | 22 | 37 | 17.54 | 12 | 203 | 60.15 | 26.7 | 0 |
| GET | /products/search | 44550 | 0 | 18 | 520 | 30000 | 456.9 | 12 | 30128 | 1469.09 | 116.7 | 0 |
| | Aggregated | 55903 | 0 | 17 | 520 | 600 | 367.67 | 12 | 30128 | 1182.96 | 143.4 | 0 |

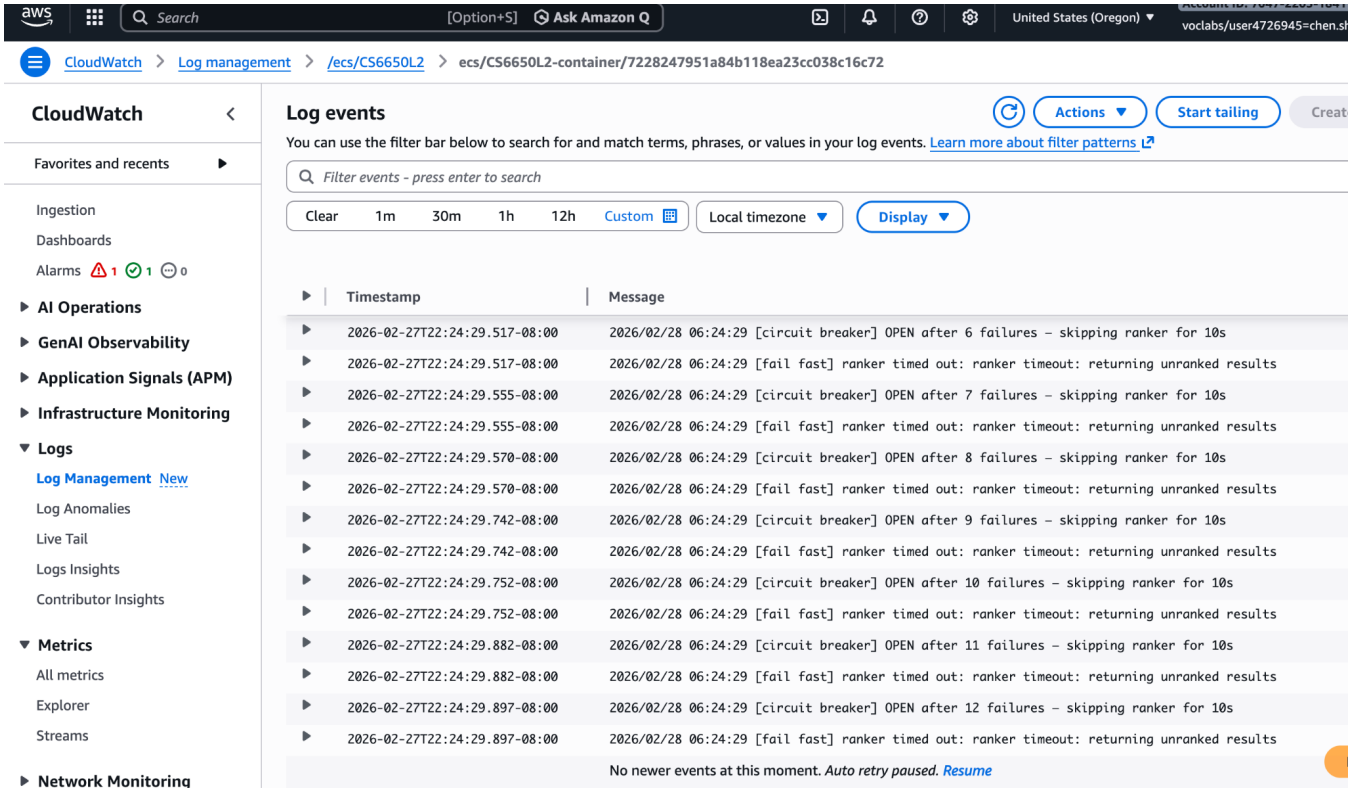


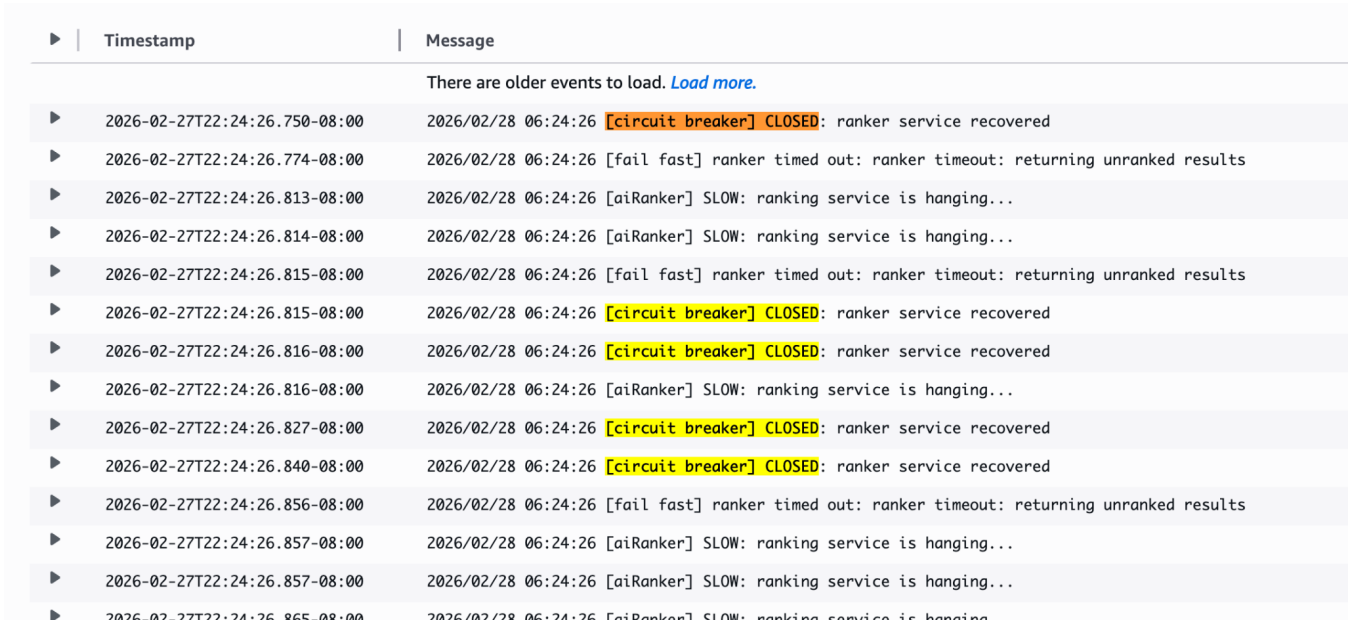


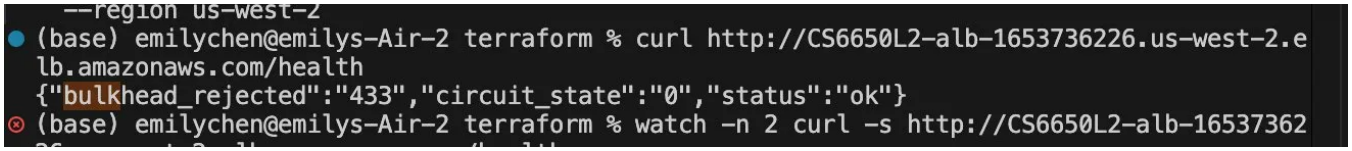

## 6. CloudWatch Log Evidence

The CloudWatch log stream for the fixed ECS task captured all three patterns firing simultaneously:

| Log Line | Pattern | Meaning |
|---|---|---|
| `[aiRanker] SLOW: ranking service is hanging...` | **Fault injection** | **30% chance triggered** |
| `[fail fast] ranker timed out: returning unranked results` | **Fail Fast** | **500ms timeout fired** |
| `[circuit breaker] OPEN after 6 failures` | **Circuit Breaker** | **Breaker tripped open** |
| `[circuit breaker] CLOSED: ranker service recovered` | **Circuit Breaker** | **Probe succeeded, closed** |
| `[bulkhead] REJECTED (total rejected: 433)` | **Bulkhead** | **Pool protected** |




## 7. Conclusions

This experiment demonstrated how a single unprotected downstream dependency can bring a healthy service to near-zero throughput under realistic load. The three Newman patterns work best in combination: the bulkhead prevents goroutine pool exhaustion, the circuit breaker stops wasting resources on a known-bad service, and fail fast ensures individual requests degrade gracefully rather than hanging indefinitely.

A key insight from the experiment: the broken version showed 0% HTTP error rate in Locust while delivering a completely degraded user experience. This highlights why SLOs based purely on error rates are insufficient — latency percentiles (P95, P99) are essential for detecting cascading failure patterns in production systems.

The ranker_status field added to the API response body — showing values of ranked, timeout_fallback, circuit_open, or bulkhead_rejected — provides real-time observability into which resilience path each request took. This pattern of exposing degradation mode in the response is a practical production technique for debugging partial failures without requiring log analysis.

***Infrastructure: AWS ECS Fargate (us-west-2) | ALB: CS6650L2-alb | Experiment date: February 27, 2026***